Lecture 4 - Move Acceptance

Created	@February 20, 2024 9:08 AM
Type	Lecture
Reviewed	☐

Stochastic Local Search – Single Point Based Iterative Search (Local Search Meta-heuristics) - revisited

```
// Algorithm 1: Outline of a Local Search Metaheuristic
1. Initialize solution
   s <- generateInitialSolution()

2. Set the best solution to the initial solution
   s_best <- s

3. Loop until termination criteria are met
   while termination criteria not met do

4. Generate a new solution based on the current solution
       s' <- apply(h, s)

5. Process the new solution with process1
       process1()

6. Decide whether to accept the new solution
       s <- acceptRejectDecision(s, s')

7. If the new solution is better than the best solution, upda
te the best solution
```

```
        if f(s').isBetterThan(f(s_best)) then
            s_best <- s'

8. Process the current solution with process2
        process2()

9. End loop
   end

10. Return the best solution found
    return s_best
```

- Stochastic Local Search - Single Point Based Iterative Search method is created using a simple framework (as discussed before):
  - Generate an initial solution
  - Have a loop that will make a change on the representation/solution we are currently handling
  - Create a new solution
  - Then decide whether to accept or reject the new solution
  - Until some termination criteria are met
- **Move Acceptance** - decides whether to accept or reject the new solution considering its evaluation/quality, $f(s')$
- In the provided pseudocode, the move acceptance is represented by the following line:

```
s <- acceptRejectDecision(s, s');
```

- **Move Acceptance (acceptRejectDecision)**: This function decides whether the new solution s' should replace the current solution s. The decision could be based on various criteria:
  - **Improvement**: If s' is better than s according to the objective function f, it is accepted. We don't have to accept the better moves every-time we can

accept similar or worsening moves sometimes

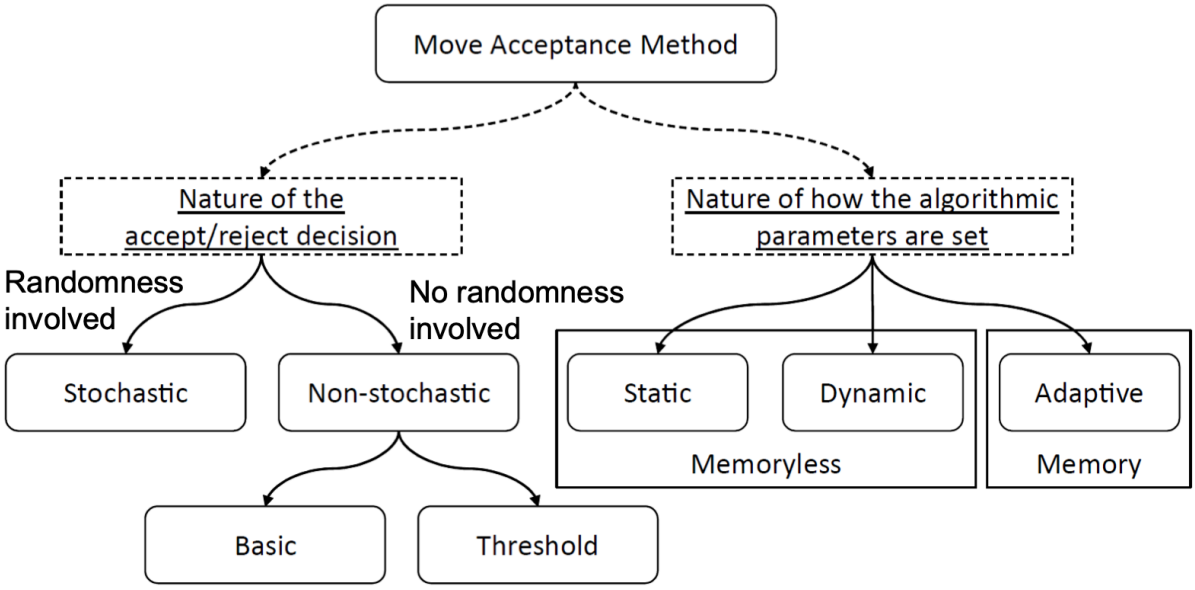
  - **Equal Quality**: If s' is of equal quality to s, it might still be accepted to maintain diversity in the search.
  - **Worse Quality**: Sometimes s' may be worse than s but is accepted with a certain probability to allow for exploration and to avoid being trapped in local optima. This is characteristic of metaheuristic algorithms like Simulated Annealing or Tabu Search.

- **Objective Function Comparison**:
  - The pseudocode contains a conditional check to see if the new solution s' has a better objective function value than the best solution found so far s_best:
    If s' is better, then s_best is updated to s'.

    ```
    if f(s').isBetterThan(f(s_best)) then
        s_best <- s';
    ```

- Move acceptance mechanisms are essential for the performance of local search algorithms. They define the balance between exploitation (searching around the current area) and exploration (moving to a different area of the solution space).
- Effective move acceptance strategies can significantly enhance the algorithm's capability to find global optima or high-quality solutions in complex landscapes.
- Accepting non-improving moves could be used as a mechanism to escape from local optimum

Breakdown of Move Acceptance



1. **Nature of the accept/reject decision** refers to whether the decision to accept a new solution in the search space is deterministic or involves some level of randomness:
    - **Stochastic (Randomness involved)**: The decision to move from one solution to another incorporates some probability. Even if the new solution is not better than the current solution, there is still a chance that it will be accepted. This is typically used to avoid getting stuck in local optima and to explore the search space more broadly.
    - **Non-stochastic (No randomness involved)**: The decision to accept a new solution is based solely on predefined rules without any randomness. If the new solution is better (or meets some acceptance criteria), it is accepted; otherwise, it is rejected. This approach can lead to rapid convergence but can also increase the risk of getting trapped in local optima.
2. **Nature of how the algorithmic parameters are set** deals with the strategy used to determine the parameters that govern the accept/reject decision and other aspects of the algorithm:
    - **Static**: Parameters are set at the beginning and remain unchanged throughout the algorithm's execution. This can lead to straightforward implementation but may not be flexible enough to adapt to different phases of the search.

    - **Dynamic**: Parameters may change during the execution based on a predefined mechanism or schedule, allowing the algorithm to shift strategies as it progresses.
    - **Adaptive (Memory)**: Parameters are adjusted based on feedback from the algorithm's performance over time. This allows the algorithm to "learn" which strategies work best and to fine-tune its behaviour for better results.
- Both of these aspects are crucial in designing a move acceptance strategy because they directly influence how the algorithm navigates the search space. A well-designed move acceptance method balances exploration (trying out less promising or diverse solutions) and exploitation (focusing on the most promising area of the search space) to effectively search for the optimal or near-optimal solutions to a problem.

▼ **Analogy**

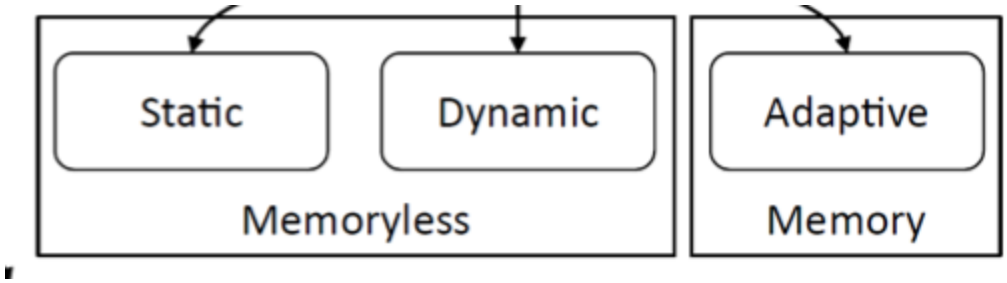
Imagine you're playing a game of hot and cold where you're trying to find a hidden treasure in a large field. The "move acceptance method" is like the strategies you use to decide which direction to walk:

1. **Nature of the accept/reject decision**:
    - **Stochastic (Randomness involved)**: This is like occasionally taking random steps in any direction, not just the direction that sounds "warmer," to explore new areas you haven't considered. This could lead you to the treasure even if it initially seems like the wrong way.
    - **Non-stochastic (No randomness involved)**: This is like choosing to only walk towards directions that definitely sound "warmer," ensuring every step is deliberate and closer to the goal, but potentially missing out on the treasure if it requires a counterintuitive step.
2. **Nature of how the algorithmic parameters are set**:
    - **Static**: This is like wearing headphones that only allow you to hear "warmer" or "colder" at a fixed volume throughout the game. You use the same strategy from start to finish.
    - **Dynamic**: This is like adjusting the volume of the headphones as you play, making it louder as you get closer to encourage smaller, more precise steps.

- **Adaptive (Memory)**: This is like keeping track of the paths you've taken. If you notice you're getting "colder," you remember which directions sounded "warmer" before and adjust your path accordingly. Your strategy evolves based on past feedback.

In the context of this game, the "move acceptance method" helps you decide whether to take a step that might initially seem "colder" (stochastic acceptance) or to always move towards "warmer" (non-stochastic acceptance). Additionally, it helps you decide if you should keep the same listening strategy (static) or change it as you get different feedback (dynamic or adaptive).

Parameter Setting Mechanisms in Move Acceptance



1. **Static Parameter Setting**:
   - In this approach, parameters are not changed throughout the execution of the algorithm. They are either absent or set to a fixed value at the start. For example, if there's a parameter named `IoM`, it could be set to 5 (`IoM=5;`) and would remain at this value during the entire algorithmic process.
2. **Dynamic Parameter Setting**:
   - Here, parameters change according to a predetermined rule that depends on the current time or the number of iterations, but not on the algorithm's performance history. So if `iter_current` represents the current iteration and `iter_max` the maximum number of iterations, `IoM` might be calculated as `IoM = round(1 + (iter_current / iter_max) * 4);`. This means that `IoM` will

increase as the algorithm progresses through its iterations, following a predefined formula.

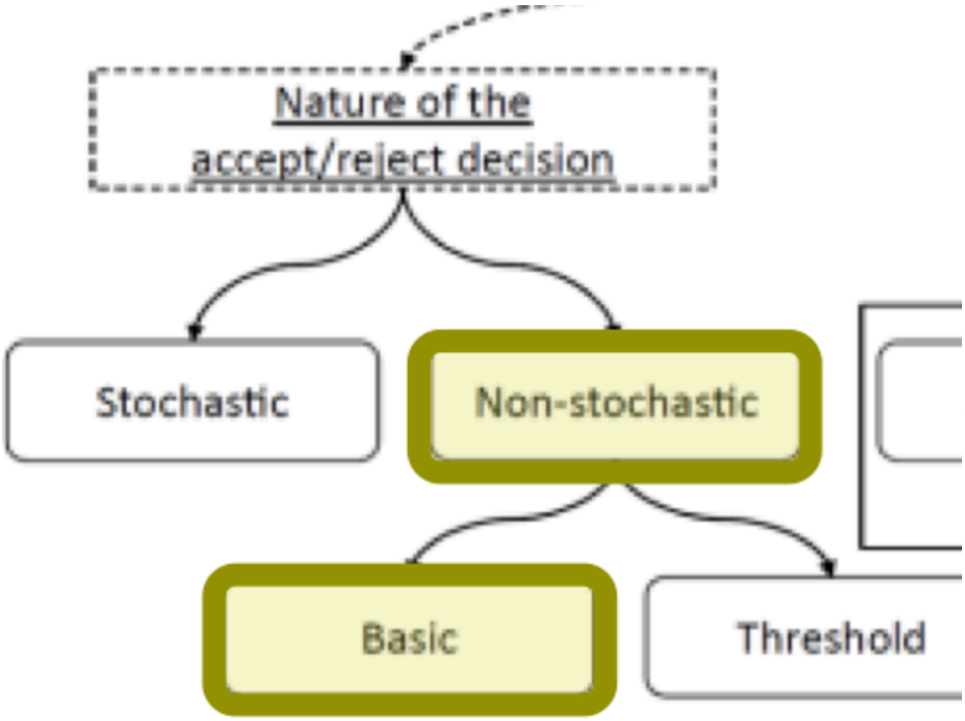
3. **Adaptive Parameter Setting**:
    - In this mechanism, parameters adjust based on the performance history of the algorithm. The adjustment is made to improve the search based on past outcomes. For instance, `IoM` might be incremented (`IoM++;`) if the algorithm has not found a better solution after 100 steps. Conversely, if a new best solution is found, `IoM` might be reset to 1 (`IoM=1;`) to reflect the updated state of the search. This adaptiveness allows the algorithm to 'learn' and improve as it processes.

These parameter setting mechanisms determine how the move acceptance criteria evolve throughout the search process:

- **Static** settings provide consistency and simplicity.
- **Dynamic** settings allow the algorithm to change its behaviour as it moves through the search space, based on the predefined progression rather than past performance.
- **Adaptive** settings enable the algorithm to use its performance history to inform future behaviour, potentially allowing it to better navigate complex search landscapes and avoid local optima.

These mechanisms are essential in defining how a metaheuristic algorithm explores the solution space and converges on an optimal or satisfactory solution. They have a significant impact on the effectiveness and efficiency of the search process.

Non-stochastic Basic Move Acceptance Methods



- **Non-stochastic Basic Move Acceptance Methods**: These methods **do not involve randomness** in the decision-making process for accepting or rejecting a move during the search for optimal solutions.
- **Reuse of Objective Values**:
  - The approach reuses the objective function values of previously encountered solutions to make accept/reject decisions.
- **Static Methods**:
  - **All Moves**: Accepts all moves without considering their impact on the objective function value.
  - **Improving Moves Only**: Accepts only the moves that improve the objective function value, i.e., $f(s') < f(s)$.
  - **Improving and Equal**: Accepts the moves that either improve or maintain the objective function value, i.e., $f(s') \leq f(s)$.
  - Here s' is the new solution.
  - s is the old solution.
  - So its basically a comparison between the new and the old solution.

- **Dynamic Methods**:
  - **None**: There are no dynamic methods mentioned for non-stochastic basic move acceptance.
- **Adaptive Methods**:
  - **Late Acceptance**: This strategy compares the quality of the current solution s' with that of the solution s_{t-L} that was accepted or visited L iterations previously. The move is accepted if the current solution is not worse than the solution from L iterations ago, i.e., $f(s') \leq f(s_{t-L})$.
  - Again s' is the new solution.
  - $s_t - L$ is a solution we acquired L steps before.

Late Acceptance (part of adaptive methods for non-stochastic basic move acceptance)

<<Overview : Compare current solution to new one or to an older one at iteration v, this makes it less greedy because it can accept improvement of old solutions which can help it escape local optima>>

```
// Late Acceptance Algorithm Pseudocode

// Initial solution generation
Produce an initial solution s
Calculate initial cost function C(s) as f(s)
Specify the history list size Lh

// Initialize the fitness array with the cost of the initial
solution
For all k in {0...Lh-1}
    f[k] := C(s)

// Initialize iteration and idle counters
First iteration I := 0
Idle iterations Idle := 0
```

```
// Main loop with termination conditions
Do until (I > 1000000) and (Idle/I > 0.02)
    // Construct a new candidate solution
    Construct a candidate solution s*
    Calculate the candidate cost function C(s*)

    // Compare candidate solution to the current solution
    If C(s*) >= C(s)
        Increment the idle iterations number Idle := Idle + 1
    Else
        Reset the idle iterations number Idle := 0

    // Calculate the index for the history based on circular
queue concept
    Calculate the virtual beginning v := I mod Lh

    // Acceptance condition based on late acceptance strategy
    If C(s*) <= f[v] or C(s*) < C(s)
        Accept the candidate s := s*
    Else
        Reject the candidate s := s

    // If the current solution improves the cost, update the
fitness array
    If C(s) < f[v]
        Update the fitness array f[v] := C(s)

    // Increment the iteration number
    Increment the iteration number I := I + 1
EndDo

// The best solution is returned
Return s
```

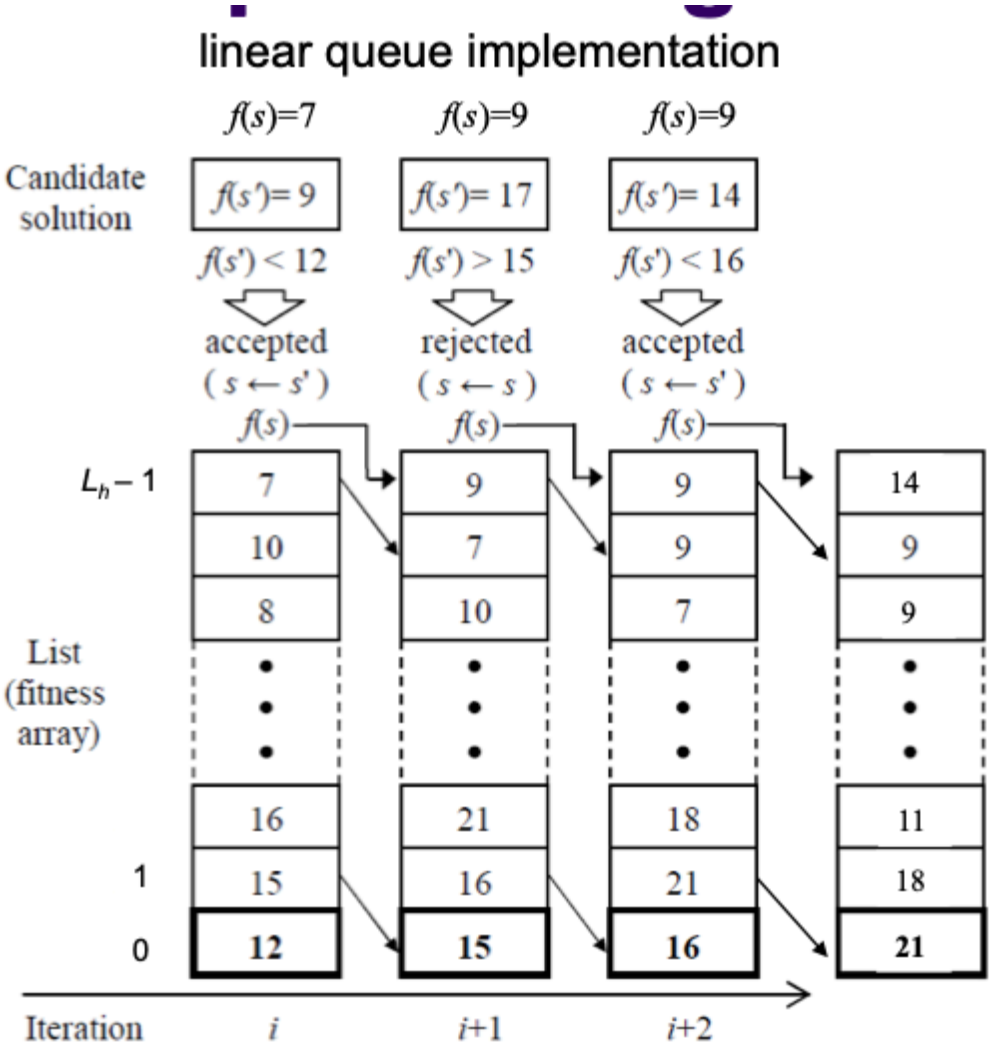
1. **Initialisation**:
    - An initial solution `s` is generated.
    - The initial cost `C(s)` is calculated, representing the objective value of `s`.
    - A list called `f` (referred to as the fitness array) is initialized with all its elements set to `C(s)`. This array holds the objective values of previously encountered solutions and is used to make acceptance decisions.
    - `Lh` specifies the length of the fitness array, effectively the memory of past solution costs.
2. **Iterative Process**:
    - The algorithm enters a loop that continues until a termination condition is met, typically a maximum number of iterations or a satisfactory idle ratio indicating too many non-improving moves.
    - A new candidate solution `s*` is constructed, and its cost `C(s*)` is calculated.
    - If the cost of `s*` is not better than `s`, an idle counter is incremented, tracking the number of non-improving iterations.
3. **Late Acceptance Criterion**:
    - The algorithm calculates an index `v`, which is the current iteration number modulo the length of the fitness array (`Lh`), to access the circular queue.
    - The new solution `s*` is accepted if `C(s*)` is less than or equal to the cost in the fitness array at index `v` or if it's better than the current solution `C(s)`. This is the "late acceptance" part, where a new solution can be accepted even if it's not better than the current solution but is better than a solution from `Lh` steps ago.
    - If `C(s)` is less than the cost at index v in the fitness array, the fitness array at this index is updated with C(s). This keeps the history current, replacing older values with newer, better ones.
    - The fitness array represents the memory of the algorithm, storing the objective values (costs) of the solutions at various past points in the search.

- By comparing the cost of the current solution with these historical values, the algorithm decides whether to accept a solution that may not improve the current state but is an improvement over a past state.
- This acceptance of non-improving moves can help the algorithm escape local optima and explore the search space more thoroughly.
- By initialising all elements of the fitness array with the same objective value, the algorithm starts with a uniform memory.
- This is essential because, at the start, there is no history of past solutions. As the algorithm progresses, this memory gets updated with the costs of better solutions.
- The late acceptance criterion relies on this memory to make informed decisions about whether to accept or reject new solutions based on past performance, not just immediate improvements.

4. **Fitness Array Update**: If the current solution improves the cost, the relevant entry in the fitness array is updated.
5. **Iteration Increment**: The iteration counter is incremented.

- **Conclusion -** After exiting the loop (once the termination conditions are met), the algorithm returns the current solution `s`, which is considered the best solution found.
- **Key Concepts:**
  - **Late Acceptance Strategy: This strategy makes the algorithm less greedy by allowing it to accept solutions that are not better than the current solution but are better than solutions from the past, enabling it to escape local optima.**
  - **Idle Iterations and Termination**: The algorithm monitors its progress and terminates when it appears to be making insufficient progress, as measured by the ratio of idle iterations.

Example walkthrough



Initial Setup

- **Queue Initialisation**: A fitness array `f(j)` is set with a length `Lh` of 1, which means it will store the cost of the most recent solution. Initially, it's filled with the cost of the initial solution, `f(s) = 7`.

Iteration i

- **Candidate Solution**: Generate new candidate solution with `f(s') = 9`.
- **Acceptance Check**: Since `f(s') < 12`, the candidate is better than the acceptance threshold and is accepted.
- **Queue Update**: The queue is updated to reflect the new current cost, so `f(j)` is now `9`.

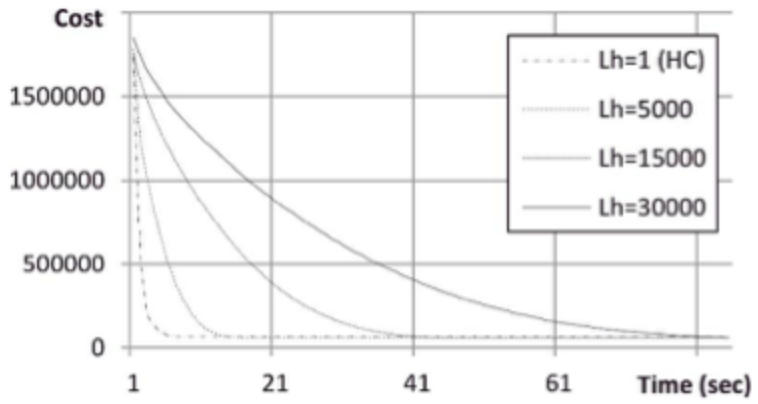
Iteration i+1

- **Candidate Solution**: Generate new candidate solution with `f(s') = 17`.
- **Rejection Check**: Since `f(s')` is not less than the queue's current cost `f(j)=9`, and it's also worse than the current solution `f(s)=9`, the candidate is rejected.
- **Queue State**: The queue remains unchanged because the new candidate was not accepted. It still contains `f(j) = 9`.

Iteration i+2

- **Candidate Solution**: Generate new candidate solution with `f(s') = 14`.
- **Rejection Check**: Again, `f(s')` is not less than the queue's current cost `f(j)=9`, nor is it better than the current solution `f(s)=9`, so the candidate is rejected.
- **Queue State**: The queue continues to hold `f(j) = 9` as no new solution has been accepted.

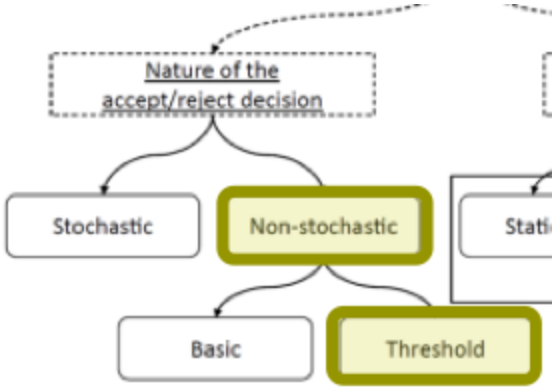
Observations for Each Iteration



- Simple approach with a single parameter, yet the setting of the list length parameter influences the overall performance of the algorithm.
- **Queue Behaviour**: The queue is acting as a memory of the best solution from `Lh` iterations ago. Since `Lh=1` in this example, it always holds the cost of the last solution.

- **Queue Role in Acceptance**: If the queue had a larger `Lh`, it would store more past costs, and we might see a different acceptance/rejection pattern as the algorithm would compare new candidates against older solutions, not just the most recent one.
- The queue is a crucial component of the Late Acceptance Algorithm, providing a mechanism to remember past solution costs, which influences the acceptance of new solutions and helps the algorithm avoid being too greedy and potentially escape local minima.
- A **longer** queue allows more previous objective function values to be stored which can result in more **exploration**
- A **shorter** queue means there is less values to compare to but can result in more **exploitation**

Non-stochastic Threshold Move Acceptance - minimising F problem



- **Non-Stochastic Threshold Move Acceptance -** In non-stochastic threshold move acceptance methods, the algorithm determines a threshold that defines a range around the current best solution's quality.
    - Any new solution that has a quality within this threshold range is accepted. This contrasts with stochastic methods that use randomisation to decide whether to accept a new solution.

- The non-stochastic approach relies on a deterministic threshold, which could be a fixed value or one that changes over time based on some criteria.
- This method enables the exploration of the solution space in a controlled manner, potentially allowing for the escape from local optima.
- Here we now make use of a threshold value in replacement of the objective value (although its still based off of it) and its used for comparison.
- Generally the threshold value is changed over time and is managed in different ways.

```
s0 = generateInitialSolution();
s, Sbest = s0;
// initialise other relevant parameters if there is any
REPEAT
    s' = makeMove(s, memory); // choose a neighbour of s*
    threshold-ε = moveAcceptance.getThreshold(s, s', memory);
    if ( f(s') <= threshold-ε ) s = s'; // else reject new so
lution s'
    Sbest = updateBest(s, Sbest); // keep track of Sbest
UNTIL(termination conditions are satisfied);
RETURN Sbest
```

1. **Initial Solution**: The algorithm begins by generating an initial solution `s0`.
2. **Initialisation**: The current solution `s` and the best solution `Sbest` are both set to `s0`. Other relevant parameters might also be initialized at this point.
3. **Repeat Loop**: The algorithm enters a loop where it will repeatedly attempt to find a better solution.
    - `makeMove(s, memory)`: It generates a new candidate solution `s'` based on the current solution `s` and possibly some memory structure that keeps track of past moves or solutions.
    - `getThreshold(s, s', memory)`: A threshold is determined based on the current solution, the candidate solution, and the memory. The `ε` represents a

small value that provides a margin for acceptance.

- `if ( f(s') <= threshold-ε )`: The algorithm checks if the candidate solution's quality is within the acceptance threshold. If it is, the candidate solution becomes the new current solution.
- `updateBest(s, Sbest)`: If the new current solution is better than the best solution found so far, update `Sbest`.

4. **Termination**: The loop continues until specified termination conditions are met (such as a maximum number of iterations or a time limit).
5. **Return Best Solution**: Once the loop terminates, the algorithm returns the best solution found, `Sbest`.

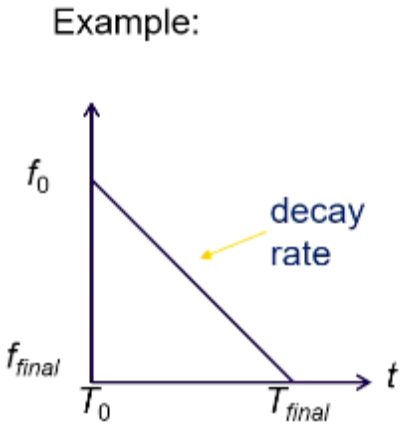
- The use of a threshold in this manner allows the algorithm to accept not only improvements but also solutions that are close in quality to the current best, which can introduce diversity into the search and avoid premature convergence.

Examples of the how the threshold value is managed (Non-Stochastic threshold move acceptance)

- **Static**: This is the simplest form of threshold acceptance. A static threshold is set, and any solution that does not worsen the objective function value by more than this fixed threshold is accepted. It's non-adaptive and does not change over time or based on the search history.
- **Great Deluge**: Under the Great Deluge algorithm, the acceptance threshold starts at a certain level and decreases over time, like water receding in a flood. Solutions are accepted if they are under the "water level," which gradually lowers, forcing the algorithm to find better solutions over time to "stay dry."
- **Flex Deluge**: This is likely a variant of the Great Deluge with a more flexible adjustment of the water level, potentially based on search progress or other criteria.
- **Record to Record Travel (RRT)**: In this approach, the threshold is dynamically adjusted based on the best solution found so far. It accepts solutions that are within a certain range of the best solution's objective value.

- **Extended Great Deluge**: It's a modified version of the Great Deluge algorithm that adjusts the water level decay rate based on feedback received during the search. If there's no improvement after a significant number of iterations, the decay rate may be reset.
- **Modified Great Deluge**: This would be another variation of the Great Deluge, likely introducing additional modifications to how the water level is managed or how solutions are accepted.

Great Deluge - minimisation



- Initially the algorithm develops a target objective value and this is changed in time/gradually decreased towards the final solution/target in a linear fashion.
- Each time we compare the objective value of the new solution to the threshold value that we obtain from a linearly decreasing function.
- The Great Deluge algorithm is a heuristic optimisation method inspired by the concept of a landscape being flooded by rising water.
- For minimisation problems, the "water level" represents an upper bound on the objective function value that will be accepted. Over time, this water level is decreased ("rain speed" or decay rate), which forces the algorithm to find solutions with increasingly lower objective function values in order to stay "above water" (i.e., remain acceptable).

The Graph Explained:

- **Vertical Axis (Objective Function Value, `f`)**: This axis represents the quality of the solutions, measured by the objective function that the algorithm is trying to minimize. Lower values on this axis represent better solutions.
- **Horizontal Axis (Time, `t`)**: This axis represents time or, in the context of an algorithm, iterations. Moving from left to right, the algorithm progresses through different solutions over time.
- **Initial Water Level (`f0`)**: At the start (`T0`), the initial water level is set, which is the threshold of solution quality that will be accepted. This is typically set at or slightly above the objective function value of the initial solution.
- **Final Water Level (`ffinal`)**: By the end of the run (`Tfinal`), the water level has decreased to this final value, meaning that the algorithm will only accept solutions of at least this quality.
- **Decay Rate**: This slope represents how quickly the water level is lowered over time. A steeper slope indicates a faster decay rate, which means the algorithm becomes more selective more quickly.

Contextual Explanation Based on the Graph:

1. **Starting Point**: At the beginning (`T0`), the water level is high, allowing the algorithm to accept a wide range of solutions, including those that may be worse than the initial solution.
2. **Water Level Decay**: As time progresses, the water level decreases steadily at the decay rate shown. The water level represents the worst solution quality that is acceptable. Solutions with a quality above the current water level (higher objective function value) are rejected.
3. **Solution Acceptance**: Each time a new solution is found that has an objective function value below the current water level, it is accepted. This is represented by the points on the graph that are below the line from `f0` to `ffinal`.
4. **Progressive Refinement**: The algorithm is forced to find progressively better solutions due to the lowering water level. Early on, it can wander through a broader landscape of solutions, but as time goes on, it must find solutions that continually improve to stay below the descending threshold.

5. **End of the Algorithm**: As the algorithm approaches `Tfinal`, the water level is at its lowest (`ffinal`), and the algorithm will only accept solutions that are of high quality (low objective function value). At this point, if the algorithm cannot find better solutions, it may terminate, returning the best found solution.

Pseudocode:

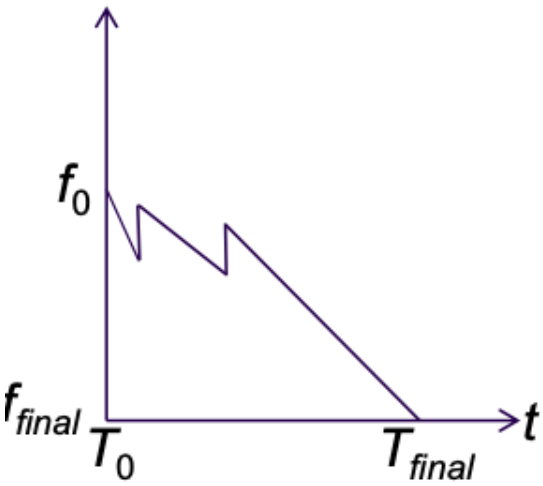
```
Choose an initial configuration
Set the initial water level and rain speed
REPEAT
    Choose a new configuration that is a stochastic perturbat
ion of the old configuration
    Compute the quality (E) of the new configuration
    IF E < water level THEN
        Accept the new configuration as the current configura
tion
        Update the water level by decreasing it by the rain s
peed
    ELSE
        Reject the new configuration
    IF no improvement for a long time THEN
        Possibly adjust parameters or stop the algorithm
UNTIL a stopping condition is met
RETURN the best configuration found
```

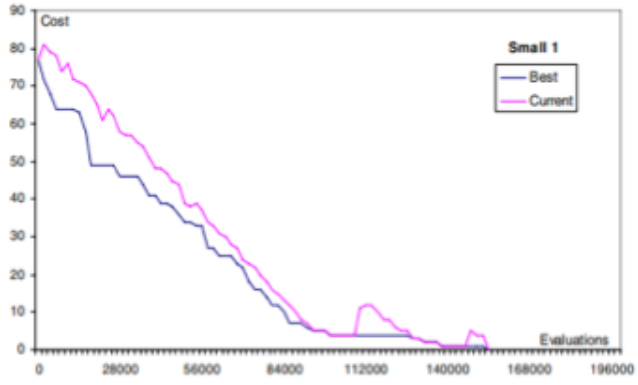
1. **Initial Configuration**: Start by choosing an initial solution for the problem.
2. **Set Initial Water Level**: Define an initial water level, which is the current threshold for the objective function's value that will be accepted.
3. **Set Rain Speed**: Determine the rate at which the water level will decrease over time.
4. **Optimisation Loop**:
    - **Generate New Solution**: Generate a new candidate solution by making a small, random change to the current solution.

- **Evaluate New Solution**: Calculate the objective function value (`E`) of the new candidate solution.
- **Acceptance Check**: If the new solution's objective function value is less than the current water level, accept it as the new current solution.
- **Update Water Level**: After accepting a new solution, decrease the water level by the predetermined rain speed.
- **Rejection**: If the new solution is not better than the water level, it is rejected, and the algorithm retains the current solution.

5. **Termination Check**:
   - If the algorithm has gone a long time without any improvement, or if too many iterations have passed, you may need to adjust the parameters or terminate the algorithm.
6. **Repeat**: Continue this process until a stopping condition is met, such as a time limit or a satisfactory solution is found.
7. **Return Best Solution**: Once the loop is terminated, the algorithm returns the best solution found.

Extended Great Deluge - minimisation





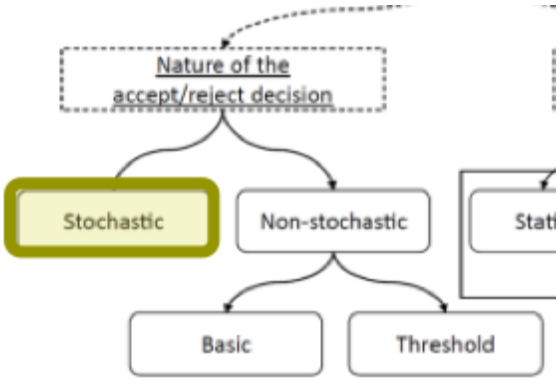
Algorithm Process:

1. Start with an initial solution and set the initial boundary level (`B`) equal to the cost function of this solution.
2. The decay rate (`ΔB`) is set based on a cooling parameter, determining how fast the water level will decrease.
3. The algorithm searches for new solutions by applying some heuristic to the current solution.
4. A new solution (`s'`) is accepted if it's equal to or better than the current solution or if it's below the current water level.
5. The water level (`B`) is lowered by `ΔB` with each accepted solution.
6. If there's no improvement over a set period (`T`), the water level is reset to the current best solution's value, and a new decay rate is set based on a secondary cooling parameter.
7. Feedback is received during the search and decay-rate is updated/reset accordingly whenever there is no improvement for a long time.

Key Points:

- Feedback during the search leads to adjustments in the decay rate.
- The water level can be reset if no improvement is found over time, which allows the algorithm to avoid stagnation.
- The algorithm becomes progressively more selective but can adapt based on the search's progress.
- The Extended Great Deluge algorithm, with its adaptive feedback mechanism, is designed to refine its search strategy over time, balancing exploration and exploitation to effectively minimise the objective function.

Stochastic Move Acceptance



- We are now going to use a random component to decide whether to accept or reject a new solution.
- Stochastic Move Acceptance introduces randomness into the decision-making process of optimisation algorithms, providing a balance between exploration (accepting worse solutions) and exploitation (accepting only better solutions).
- The strategy is designed to prevent the algorithm from becoming trapped in local optima and to explore a broader search space. Different variations of this strategy adjust the acceptance probability according to different rules to suit various optimisation problem scenarios.

```
s0 = generateInitialSolution ;
s, sbest= s0;
REPEAT
s' = makeMove(s, memory); // choose a neighbour of s
P = moveAcceptance-getAcceptanceProbability(s, s', memory);
r = getRandomValue(); // e.g., a uniform random value ∈[0,1)
if ( f(s').isBetterThan( f(s) ) || (r < P) )
s = s'; // else reject new solution s'
sbest ← updateBest s, s´ ; // keep track of sbest
UNTIL (termination conditions are satisfied);
RETURN sbest;
```

- **Initial Solution**: The process begins by generating an initial solution, `s0`.
- **Best Solution Tracking**: The algorithm keeps track of the best solution found so far, `s_best`.

- **Repeat Loop**: The main loop involves generating new solutions, calculating their acceptance probability, and deciding whether to accept them.
- **New Solution Generation**: A new solution `s'` is created by making a random change to the current solution `s`.
- **Acceptance Probability**: For each new solution, an acceptance probability `P` is calculated, which may depend on the current solution, the new solution, and any historical information stored in `memory`.
- **Random Decision**: A random number `r` is drawn, and if it is less than or equal to `P`, or if the new solution is better than the current solution, `s'` is accepted.
- **Update Best Solution**: If the new solution is better than `s_best`, update `s_best`.
- **Termination**: The loop continues until a stopping condition is met, such as a maximum number of iterations or time limit.
- **Result**: Returns the best solution found, `s_best`.

Stochastic Move Acceptance Examples:

- **Static**: Naive Acceptance where `P` is a fixed probability. If a new solution improves the objective function, it is accepted with probability `P=1.0`, otherwise, it might still be accepted with a lower probability (e.g., `P=0.5`).
- **Dynamic**: Simulated Annealing where `P` changes over time in relation to the difference in the quality of the current and previous solutions. The 'temperature' parameter decreases over time, reducing the probability of accepting worse solutions as the algorithm proceeds.
- **Adaptive**: Simulated Annealing with reheating adjusts `P` by occasionally increasing the temperature, which can help the algorithm escape from local optima by temporarily increasing the probability of accepting worse solutions.

Simulated Annealing

- Simulated Annealing is a **probabilistic** technique for approximating the global optimum of a given function. It is a type of **stochastic local search** algorithm and is metaphorically similar to the physical process of annealing in metallurgy.

- **Inspiration**: Simulated Annealing is modelled after the process of heating and then slowly cooling a material to decrease defects and increase the size of the crystal domains.
- **Ease of Implementation**: The algorithm is straightforward to implement.
- **Performance**: Given enough time, it tends to perform well because it can escape local optima by allowing worsening moves.
- **Parameter Sensitivity**: The performance of the algorithm is highly dependent on the settings of its parameters, such as the cooling schedule.
- **Theoretical Properties**: It has well-studied theoretical underpinnings, particularly regarding convergence properties, though these may not always translate into practical benefits.

Analogy for Simulated Annealing:

- **Cooling Material**: If a liquid cools and solidifies too quickly, it may form a suboptimal crystalline structure. Similarly, if an optimisation algorithm converges too fast, it might get stuck in a local optimum.
- **Slow Cooling**: By cooling slowly, a material forms a more orderly crystal structure, minimizing energy states. In optimisation, slowly reducing the "temperature" of the search allows the algorithm to explore the solution space more thoroughly and increases the chances of finding a global optimum.

How Simulated Annealing Works:

1. **Initial Solution**: Start with an initial solution to the optimisation problem.
2. **Temperature**: Set an initial "temperature" that is high enough to allow the algorithm to accept worse solutions with relatively high probability.
3. **Iterative Process**:
    - Generate a new solution by making a small, random change to the current solution.
    - Calculate the change in the cost function between the new and the current solution.

- Accept the new solution with a probability that depends on the difference in cost and the current temperature.

4. **Cooling Schedule**: Gradually lower the temperature according to a predetermined cooling schedule, which reduces the probability of accepting worse solutions over time.
5. **Termination**: Continue this process until the system "freezes" at a minimum energy state, or in other words, a stopping condition is met (like a maximum number of iterations or temperature threshold).
6. **Result**: The algorithm returns the best solution found.

- In practice, Simulated Annealing is a balance between exploration (accepting random solutions) and exploitation (narrowing down to the best solution). The cooling schedule is crucial as it determines the rate at which the search space is explored and how quickly the algorithm begins to focus on exploitation over exploration.

Simulated Annealing Local Search Metaheuristic

```
INPUT: T 0, T final
s0 ← generateInitialSolution ;
T ← T 0; // initialise temperature to T 0
sbest ← s0; s ← s0; // set s and sbest to initial solution
REPEAT
s′ ← perturbation s ; // choose a neighbouring solution of s
Δ = f s′ − f s ;
r ← random ∈ 0,1 ; // get a uniform random number in the range
[0,1)
if(r < P (Δ, T) ) { // accept s′with Boltzmann probability
s ← s′;
}
sbest ← updateBest ; // keep track of best solution
T ← coolTemperature ; // decrease the temperature according to co
oling schedule
```

```
UNTIL (Termination conditions are satisfied);
Return sbest;
```

Pseudocode Explanation:

1. **Input Parameters**:
   - `T0`: The initial temperature, which determines the likelihood of accepting worse solutions at the start.
   - `Tfinal`: The final temperature, at which point the algorithm will stop.
2. **Initial Setup**:
   - `s0`: Generate an initial solution.
   - `T`: Set the current temperature to `T0`.
   - `s_best`: Initialise the best solution found so far to the initial solution `s0`.
   - `S`: Set the current state to the initial solution `s0`.
3. **Main Loop** (Repeat until termination conditions are met):
   - `s'`: Generate a new solution by making a random perturbation to the current solution `s`.
   - `Δ`: Calculate the change in cost between the new solution `s'` and the current solution `s`.
   - `r`: Generate a random number between 0 and 1.
   - **Acceptance Criteria**: If the new solution is better (`Δ < 0`) or if the random number `r` is less than the acceptance probability `P(Δ, T)`, update the current state to `s'`. This acceptance probability typically follows the Boltzmann distribution and decreases as the temperature `T` is lowered or as the change in cost `Δ` increases.
   - `s_best`: Update the best solution if the new solution is an improvement.
   - `T`: **Decrease the temperature according to the cooling schedule, which gradually lowers the chance of accepting worse solutions.**
4. **Termination**:

- The algorithm repeats the main loop until certain termination conditions are satisfied, such as reaching the final temperature `Tfinal` or a maximum number of iterations.

5. **Result**:
    - Return the best solution found `s_best`.

How Simulated Annealing Works in this Context:

- **Exploration vs. Exploitation**: Initially, the high temperature allows the algorithm to explore the solution space widely, including accepting moves to worse solutions which help in escaping local optima. As the temperature decreases, the algorithm gradually shifts from exploration to exploitation, focusing on refining the best solution found so far.
- **Cooling Schedule**: The rate at which the temperature decreases is critical. Too fast, and the algorithm may not adequately explore; too slow, and the algorithm may take unnecessarily long to converge.
- **Random Acceptance**: The random acceptance of worse solutions is a key feature that differentiates SA from greedy algorithms. The probability of accepting a worse solution decreases both with worsening Δ and with a lowering temperature `T`.
- **Convergence**: While the algorithm is designed to converge to a global optimum given enough time and an appropriate cooling schedule, in practice, it often converges to a solution that is good enough, if not optimal.
- In summary, Simulated Annealing is a metaheuristic that mimics the cooling process of metals to find solutions to optimisation problems.
- It strikes a balance between random exploration of the search space and gradual focusing on areas of the space that contain high-quality solutions.

Accepting Moves using SA

- **Improving Moves**: Any move that results in a lower cost (improved solution) is always accepted.

- **Worsening Moves**: Moves that result in a higher cost (worsened solution) can also be accepted, but the probability of acceptance is based on the Metropolis criterion, which depends on the current temperature `T`.
- **Metropolis Criterion**:
  - `Δ = F(S_new) - F(S_old)`: Represents the change in the cost function when moving from the current solution `S_old` to a new solution `S_new`.
  - For minimisation, a worse solution means `Δ > 0`.
  - The probability of accepting a worse solution is calculated as $P(\Delta, T) = e^{(-\Delta/T)}$.
  - A random number `U` from a uniform distribution in the range [0,1] is generated. If `U < P(Δ, T)`, the worse solution is accepted.

Explanation:

- The SA algorithm accepts all improving moves to decrease the cost function `F`.
- However, it also accepts some worsening moves to escape local minima, with a probability that decreases as either the worsening of the solution (`Δ`) increases or the temperature (`T`) decreases.
- When `T` is high, the exponential term `e^(-Δ/T)` is closer to 1, meaning there's a high likelihood of accepting even significantly worse solutions.
- As `T` decreases, the term `e^(-Δ/T)` gets smaller, especially for large `Δ`, meaning the algorithm becomes more selective, tending towards only accepting improving moves or moves that do not significantly worsen the solution.

Cooling/Annealing:

- **Temperature Decrease**: The temperature `T` is gradually lowered according to a cooling schedule.
- Initially, `T` is high, which allows the algorithm to accept many inferior solutions, aiding in global search and avoiding local optima.

- As `T` decreases, the acceptance of inferior moves becomes less likely, making the algorithm more "greedy" towards better solutions.
- **Probability of Acceptance**:
  - The table shows how the acceptance probability `P(Δ, T)` decreases as `T` decreases or as `Δ` increases.
  - For example, with `Δ=1` and `T=10`, the probability of acceptance might be `0.9`, but with `Δ=1` and `T=0.1`, the probability plummets to `0.00005`.

Explanation:

- In the beginning, SA behaves like a random search due to the high acceptance probability of various moves. This allows the algorithm to explore the solution space without getting trapped in local optima.
- Over time, as `T` decreases, the algorithm becomes more selective, converging towards exploiting the regions of the search space that have shown promise. The cooling schedule, which dictates how quickly `T` decreases, is crucial to balancing the exploration and exploitation phases of SA.
- The goal is to allow sufficient exploration at higher temperatures and then fine-tune the search around the best-found solutions as the temperature lowers, eventually converging to an optimal or near-optimal solution.

SA – Cooling Schedule

Starting Temperature:

- Should be high enough to allow acceptance of almost all neighbouring solutions early on, promoting extensive search space exploration.
- It shouldn't be so high that the search becomes a purely random search.
- A good starting temperature can be estimated by finding a temperature at which a certain percentage (like 60%) of worsening moves are accepted.

Final Temperature:

- Often set to zero, but in practice, it doesn't need to be exactly zero.

- The lower the temperature, the less likely the algorithm is to accept worsening moves, until it behaves like a greedy algorithm at or near zero.
- The algorithm stops either at a low enough temperature where no worse moves are accepted or based on other criteria, such as a maximum number of iterations or time elapsed.

Temperature Decrement Methods:

- **Linear**: The temperature is decreased by a fixed amount each iteration: `T = T - x`.
- **Geometric**: The temperature is decreased by a factor each iteration: `T = T * α`, where α is typically between [0.9, 0.99].
- **Lundy Mees**: The temperature is decreased using the formula `T = T / (1 + βT)`, where β is a small value close to zero. This schedule decreases the temperature very slowly.

Iterations at Each Temperature:

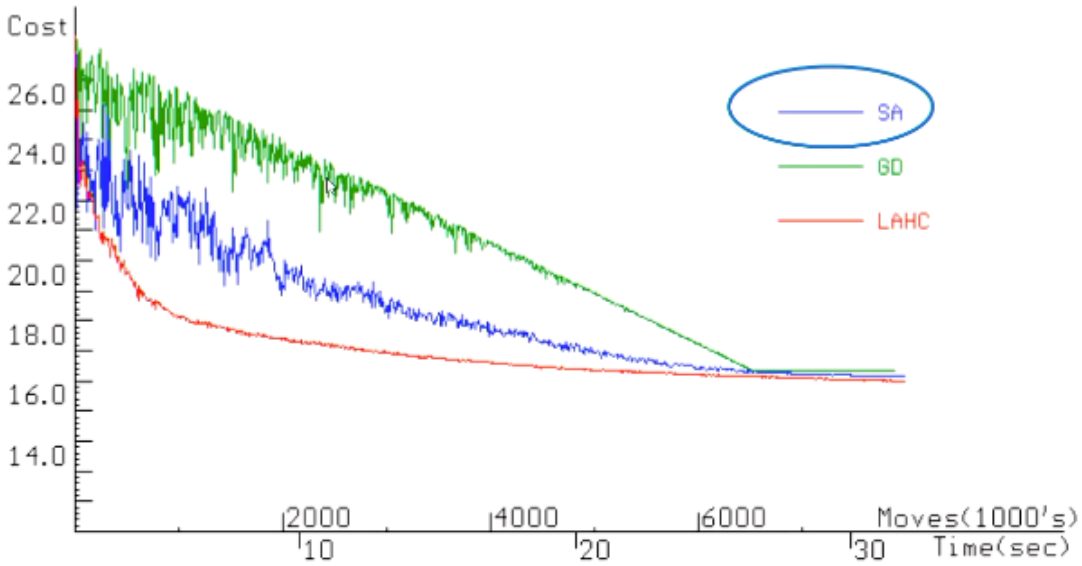
- The number of iterations at each temperature level can be fixed, or it can vary dynamically.
- A compromise between the number of temperatures to visit and the number of iterations per temperature can be made.
- Dynamically changing the number of iterations can allow more iterations at lower temperatures to exploit promising regions of the search space thoroughly.
- **Reheating**: If the algorithm is stuck at a local optimum, the temperature can be temporarily increased to allow the algorithm to escape.

Explanation of Cooling Schedule in SA:

- The cooling schedule in SA is the strategy used to lower the temperature during the optimisation process.
- It is crucial because it governs the transition from exploration to exploitation. At high temperatures, the algorithm is more explorative, accepting even highly inferior solutions, which helps it to escape from local minima.

- As the temperature decreases, the algorithm gradually becomes more exploitative, accepting only improvements or minor deteriorations, focusing on refining the best solution found.
- Each decrement method has its characteristics:
    - **Linear**: Simple but can be inefficient as it might cool too quickly or too slowly.
    - **Geometric**: More controlled and often used in practice; it cools the system exponentially.
    - **Lundy Mees**: Provides a very slow cooling process, which can be beneficial for tough optimisation problems where more gradual exploration is needed.
- The number of iterations at each temperature level is another aspect that can be tuned. More iterations at a particular temperature allow the algorithm to search more thoroughly around the current area in the solution space. Reheating can be used to jump out of local minima by temporarily increasing the temperature, reintroducing some level of exploration.
- In the context of SA, the cooling schedule is vital because it impacts the efficiency and effectiveness of the algorithm in finding a solution close to the global optimum. It balances the algorithm's convergence speed with its ability to thoroughly explore the solution space.

Some behavioural properties



- This graph illustrates the difference in behaviour between different move acceptance criteria.
- Its a simple examination timetabling problem (here its for a single run of a given problem instance and late acceptance hill climbing performs best here).
- **SA (Simulated Annealing)**: The blue line shows the cost decreasing in a somewhat noisy pattern, which is typical for SA. This noise indicates that SA is willing to accept worsening moves, especially early on when the temperature is higher. As the temperature decreases, the acceptance of worsening moves becomes less frequent, and the algorithm starts to fine-tune around the best solutions found, hence the noise reduction and the stabilization of the cost value as time progresses.
- **GD (Great Deluge)**: The green line indicates a more linear and gradual decline in cost. This suggests that GD's threshold for accepting worsening moves is steadily decreasing. Initially, GD may behave similarly to SA by accepting many inferior solutions, but as the "water level" lowers, the acceptance of such solutions becomes less common, leading to a more consistent and steady descent towards better solutions.
- **LAHC (Late Acceptance Hill Climbing)**: The red line shows the best performance among the three, with a consistent and steady descent to the lowest cost. LAHC appears to be more effective in this instance at finding

good solutions quickly and improving upon them consistently over time. The method seems to balance well between exploration and exploitation without the same level of noise seen in SA, suggesting that it may be using a more selective acceptance criterion that still allows for escape from local optima.

Additional Observations:

- **Early Exploration**: All three methods initially explore the solution space, with SA and GD potentially accepting worse solutions. This is crucial for avoiding premature convergence on local optima.
- **Convergence Behaviour**: SA and GD show convergence as they refine their search. SA, in particular, shows a decrease in the noise level as the algorithm progresses, indicating a reduction in the temperature and, consequently, in the willingness to accept worse solutions.
- **Performance Over Time**: LAHC seems to provide a better or more consistent reduction in cost over time for this problem instance, suggesting it may be more efficient for this specific type of problem.
- **Stochastic vs. Deterministic**: SA is inherently stochastic and can show variability in its performance, while LAHC, although it may have stochastic elements in the neighbourhood search, follows a more deterministic rule for accepting moves, which might account for its smoother performance curve.
- In summary, the behaviour of each algorithm is reflective of its underlying mechanics: SA's probabilistic acceptance of worsening moves early on and gradual focusing, GD's linear threshold decrease, and LAHC's consistent improvement indicate different strategies in navigating the search space to minimise the cost function in an examination timetabling problem.

Simulated Annealing with Geometric Cooling

```
INPUT: T0 (> 0), α (cooling rate <= 1.0)
Generate an initial solution S0 using some heuristics
Set Sk = Sbest = S0, k=0;
```

```
REPEAT
    Select Snew ∈ N(Sk)  // Make a move from Sk to Snew based
on N
    If F(Snew) < F(Sk) then Sk+1 = Snew  // an improving move
is made
    else  // A worsening solution is obtained
        generate a random uniform number in (0,1), U(0,1)
        If U(0,1) < e^(-(F(Snew) - F(Sk)) / Tk)
            then Sk+1 = Snew  // Accept worsening move
        else
            Sk+1 = Sk  // Reject worsening move
    // Keep track of the best solution found so far
    If F(Snew) < F(Sbest) then Sbest = Snew
    Tk+1 = α * Tk  // geometric cooling: multiply previous te
mp with α (value < 1.0)
    k = k + 1;  // one iteration at each T
UNTIL (stopping condition is true)
Return Sbest;
```

- **Initialisation**: The algorithm starts by generating an initial solution `S0` and sets the current solution `Sk` and the best solution `Sbest` to this initial solution. The iteration counter `k` is set to 0, and the initial temperature `T0` is provided.
- **Repeat Loop**: The main loop of the algorithm executes repeatedly until a stopping condition is met (such as a maximum number of iterations or a minimum temperature reached).
    - **Neighbourhood Move**: A new candidate solution `Snew` is generated from the neighbourhood of the current solution `Sk`.
    - **Improvement Check**: If `Snew` has a lower cost `F` than `Sk`, indicating an improvement, it becomes the new current solution `Sk+1`.
    - **Acceptance Probability for Worsening Move**: If `Snew` is worse than `Sk`, the algorithm generates a random number `U(0,1)` and compares it to the acceptance probability, which is based on the exponential function

$e^{(-(F(S_{new})-F(Sk))/Tk)}$. If the random number is less, the worsening move is accepted.

- **Best Solution Update**: If the new solution `Snew` is better than the best solution found so far `Sbest`, update `Sbest`.
- **Geometric Cooling**: The temperature `Tk` is updated by multiplying it by the cooling rate `α`, which should be less than 1. This geometric cooling decreases the temperature and hence the probability of accepting worsening moves as the algorithm proceeds.
- **Iteration Increment**: The iteration counter `k` is incremented.

- **Termination**: The loop continues until a defined stopping condition is satisfied.
- **Return Best Solution**: Once the loop terminates, the algorithm returns the best solution found `Sbest`.
- The geometric cooling schedule (`Tk+1 = α * Tk`) is a critical aspect of this algorithm, as it defines how quickly the algorithm becomes more conservative in accepting new solutions, especially worsening ones. The cooling rate `α` determines the rate of this temperature decrease; a smaller `α` leads to a faster decrease in temperature, making the algorithm less likely to accept worsening moves as iterations continue.

Simulated Annealing with Geometric Cooling MAX-SAT Problem Instance walkthrough

MAX-SAT Problem for Simulated Annealing Exercise:

Problem Instance:

- Formula: (a ∨ b) ∧ (¬d ∨ f) ∧ (¬a ∨ c) ∧ (b ∨ ¬f) ∧ (¬b ∨ c) ∧ (c ∨ e)
- Objective Function: Minimise the number of unsatisfied clauses.

Simulated Annealing Parameters:

- Initial Solution (`S0`): `100100`
- Initial Temperature (`T0`): `0.9`

- Cooling Rate (`α`): `0.9`

Algorithm Steps:

- Perform 3 Simulated Annealing iterations.
- Neighbourhood Operator: Random 1-bit flip in the truth assignment.
- Random numbers for bit flip decisions: `<2, 3, 1, 6, ...>`
- Random numbers for acceptance of new solutions: `<0.47, 0.089, ...>`

Methodology:

1. Start with the initial solution and temperature.
2. For each iteration:
   a. Flip a bit in the current solution according to the random numbers provided.
   b. Calculate the new cost by counting the number of unsatisfied clauses.
   c. Determine if the new solution is accepted by comparing a random number to the Boltzmann probability factor.
   d. Update the best solution if the new solution is better.
   e. Update the temperature by multiplying it with the cooling rate `α`.
3. Repeat for a total of 3 iterations or until another stopping condition is met.

Objective - minimisation:

- The goal is to find a truth assignment that satisfies the most clauses (or equivalently, leaves the fewest clauses unsatisfied) in the given MAX-SAT problem.

Initial Setup:

- The algorithm starts with an initial solution `S0 = 100100`, which is also the best solution `Sbest` so far.
- The initial temperature is set to `T0 = 0.9`.
- The cost function `F(S0)` equals `3`, indicating that there are three clauses unsatisfied by this initial solution. The clauses that are satisfied are labeled `012345`.

Iteration k=0:

- A new candidate solution `Snew` is generated by flipping a bit in the current solution `S0`. The bit to flip is chosen based on a sequence of random numbers, with the first number being `2`, indicating the second bit is flipped, resulting in `Snew = 110100`.
- The cost function `F(Snew)` is calculated as `4`, which is higher than `F(S0)`, meaning the new solution is worse since it satisfies fewer clauses.
- A random number `U(0,1) = 0.47` is generated. The acceptance probability for the worsening move, calculated using the Boltzmann factor, is `0.33=e^(-(-4-3/0.9))` . Since `0.47 > 0.33`, the move to `Snew` is rejected, and the current solution remains `S1 = 100100`. The best solution `Sbest` also remains unchanged since `Snew` was not an improvement.
- The temperature is updated using the geometric cooling formula `T1 = α * T0`, resulting in `T1 = 0.9 * 0.9 = 0.81`.

Iteration k=1:

- Another candidate solution `Snew` is generated, this time resulting in `Snew = 101100`, using the random sequence to choose the bit to flip.
- This new solution has a cost function `F(Snew) = 1`, which is lower than `F(S1)`, indicating an improvement as it satisfies more clauses.
- Since `F(Snew)` is better, the algorithm accepts the new solution, setting `S2 = Snew = 101100`.
- The best solution `Sbest` is updated to this new solution because it has a lower cost function value than the previous best, and `F(Sbest)` is now `1`.
- The temperature is again updated: `T2 = α * T1 = 0.9 * 0.81 = 0.729`.

Explanation of Decisions:

- **Improving Moves**: When `Snew` leads to a lower cost function (satisfying more clauses), it is automatically accepted, and `Sbest` is updated if `Snew` also outperforms the current best solution.

- **Worsening Moves**: If `Snew` leads to a higher cost function (satisfying fewer clauses), a random number is compared to the Boltzmann probability to decide on acceptance. Moves are more likely to be accepted if the random number is lower than the calculated probability, which would indicate the system is "willing" to explore potentially beneficial regions of the solution space by accepting temporary deteriorations.
- **Geometric Cooling**: After each iteration, the temperature is decreased by multiplying with the cooling rate `α`, which reduces the likelihood of accepting worsening moves in subsequent iterations. This systematic reduction simulates the annealing process where the material slowly settles into a state of minimum energy – in this case, a solution with the maximum number of satisfied clauses

Iteration k=2:

- **Neighborhood Move**: The algorithm attempts to move from the current solution `S2` by flipping a bit. According to the sequence of random numbers `<2, 3, 1, 6, ...>`, the second bit is selected, resulting in a candidate solution `Snew = 001100`.
- **Objective Function Evaluation**: The new solution `Snew` is evaluated to have a cost `F(Snew) = 2`, which is worse than `F(S2) = 1` because `F(Snew)` represents the number of unsatisfied clauses, and we want to minimise this value in a MAX-SAT problem.
- **Worsening Solution Acceptance Check**: The acceptance probability of this worse solution is calculated using the Boltzmann factor `e^(-(F(Snew) - F(S2)) / T2) = e^(-1 / 0.729) ≈ 0.253`. A random number `U(0,1) = 0.089` is generated, which is less than `0.253`.
- **Decision to Accept Worsening Move**: Because the random number `0.089` is less than the acceptance probability `0.253`, the move to the worse solution `Snew` is accepted, resulting in `S3 = Snew = 001100`.
- **Best Solution Check**: Since `F(Snew)` is not better than `F(Sbest)`, the best solution `Sbest` remains unchanged at `101100`.
- **Temperature Update**: The temperature is updated using the geometric cooling schedule: `T3 = α * T2 = 0.9 * 0.729 ≈ 0.6561`.

Explanation of Iteration k=2:

- **Random Bit Selection**: The bit to flip is selected based on a predetermined random sequence to introduce stochasticity in the search.
- **Acceptance of a Worsening Move**: Despite the new solution being worse, it is accepted because the SA algorithm allows for occasional acceptance of suboptimal moves to avoid getting trapped in local optima. This is particularly useful in combinatorial optimization problems like MAX-SAT, where the landscape is rugged with many local optima.
- **Temperature's Role**: As the temperature `T2` is still relatively high, there is still a reasonable probability of accepting worsening moves, facilitating exploration. However, as `T3` is now lower, subsequent acceptance of worse solutions will become less likely, leading the algorithm to focus more on exploitation of the current region of the solution space.
- **Boltzmann Probability**: The probability depends on the change in cost (number of unsatisfied clauses) and the current temperature. Since the worsening is small (`ΔF = 1`), and the temperature has not yet been lowered significantly, the algorithm is still in a phase where it is willing to explore.
- **Unchanged Best Solution**: `Sbest` does not change because the new solution did not improve the number of satisfied clauses compared to the best solution found so far.

Parameter Setting Issues and Tuning Methods

Metaheuristic Type	Algorithm	Example Parameters
Local Search	SA (Simulated Annealing)	T0 (initial temperature), α (cooling rate)
iteratively improve a single candidate solution by exploring its local neighborhood. These methods move from one solution to another by making small changes, aiming to	TS (Tabu Search)	Tabu list size (tabu tenure)

Metaheuristic Type	Algorithm	Example Parameters
find better solutions in the vicinity of the current one.		
	GLS (Guided Local Search)	λ (intensification control), a (coefficient)
	ILS (Iterated Local Search)	perturbation, perturbation strength
	VNS (Variable Neighbourhood Search)	kmin, kmax (smallest, largest neighbourhood size)
Population-based	GA (Genetic Algorithm)	population size, mutation probability, mutation
maintain and evolve a population of candidate solutions simultaneously. These algorithms use interactions among the population members to guide the search process. Examples include:	GP (Genetic Programming)	(Parameters likely similar to GA)
	EC (Genetic and Evolutionary Computation)	(Parameters likely similar to GA)
	MA (Memetic Algorithm)	(Parameters likely encompass those from GA and local search)
	PSO (Particle Swarm Optimisation)	number of particles, Vmax (maximum velocity)
Constructive	ACO (Ant Colony Optimisation)	weight of pheromone, evaporation rate
build solutions from scratch by adding components one by one in a systematic way, usually based on some greedy criterion or	GRASP (Greedy Randomised	restricted candidate list parameter

Metaheuristic Type	Algorithm	Example Parameters
heuristic. These methods are often used to generate initial solutions for other metaheuristics or to solve combinatorial problems. Examples include:	Adaptive Search Procedure)	

- Regardless of the categorising of metaheuristics we can observe that they come with certain parameters.
- Take simulated annealing:
  - What should the initial temperature be?
  - What should the cooling rate be?
  - Whats the best cooling rate for a given problem?
- For Tabu Search:
  - Whats the best tenure rate?
- Etc etc....

Parameter Types

Parameter Types for Metaheuristics:

1. **Categorical/Symbolic/Structural Parameters**:
   - These parameters include choices that define the structure or strategy of the algorithm, such as:
     - Which method to use for initialising the algorithm (e.g., random initialisation, greedy approach).
     - The type of mutation operation in a genetic algorithm (e.g., bit-flip, swap).
2. **Ordinal Parameters**:
   - These are parameters that have a meaningful order but not necessarily a fixed numerical distance between them. Examples include:

    - Sizes of neighbourhoods used in local search algorithms, categorised qualitatively (e.g., small, medium, large).

3. **Numerical/Behavioural Parameters**:
    - These parameters are quantifiable and have a direct numerical value, such as:
        - Integer or real-valued parameters that specify exact quantities or rates (e.g., population size in a genetic algorithm, evaporation rate in ant colony optimisation).
    - The actual values for numerical parameters may depend on the specific settings chosen for categorical or ordinal parameters. For instance, a "large" neighbourhood size in ordinal terms might correspond to a specific number like 100 in numerical terms.

Summary

- **Categorical parameters** are used to select algorithmic components or features.
- **Ordinal parameters** provide a ranked but not quantitatively measured configuration option.
- **Numerical parameters** offer precise control over the algorithm's behavior and are often critical to the performance of the metaheuristic.
- When configuring a metaheuristic, it's essential to consider the interplay between different types of parameters, as the effectiveness of numerical parameters might be influenced by the choices made in categorical and ordinal parameters.

Parameter Tuning

Parameter Setting Methods for Metaheuristics:

Parameter Tuning:

- **Parameter Tuning** is a critical pre-execution step that can dictate the success of the algorithm. It often requires experimentation and can be time-consuming

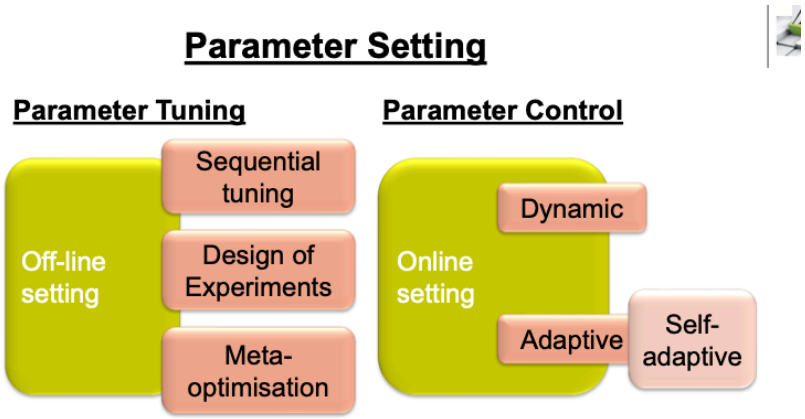
but aims to find a robust set of parameters that will perform well across different instances of the problem.

- **Definition**: Identifying the best initial settings for a set of parameters before the search process begins, which is known as the off-line phase.
- **Example**: Setting the mutation strength or mutation probability in genetic algorithms before the search starts.
- **Impact**: The initial parameter settings significantly influence the performance of a metaheuristic algorithm.
- Here you do some trials with your algorithm and decide what parameter set is best and then apply these to unseen problem instances.

Parameter Control:

- **Parameter Control** allows the algorithm to react to the search's progression and adapt its behaviour accordingly. This can be particularly beneficial in dynamic environments or when the search landscape is complex and a static parameter setting may not be optimal throughout the entire search.
- **Definition**: Adjusting the settings of parameters during the search process, which is referred to as the on-line phase.
- **Nature**: This can be dynamic, adaptive, or self-adaptive, meaning the parameters can change based on the current state of the search.
- **Example**: Modifying the mutation strength in Iterated Local Search (ILS) or the mutation probability in genetic algorithms as the search progresses.
- **Advantage**: By controlling parameters on-line, the system can potentially become less sensitive to its initial parameter settings, adapting to the search's needs.
- The algorithms will determine what parameters are best as they change in time.

A Classification of Parameter Setting



Off-line Setting (Parameter Tuning):

1. **Sequential Tuning**: Adjusting parameters one after the other in a systematic sequence.
2. **Design of Experiments (DoE)**: Using statistical methods to plan and conduct experiments to efficiently explore the parameter space.
3. **Meta-optimisation**: Applying optimisation techniques to find the best parameter settings for the metaheuristic itself.

Online Setting (Parameter Control):

1. **Dynamic**: Parameters change during the optimisation process, but the changes are not based on the performance of the algorithm.
2. **Adaptive**: Parameters are adjusted in response to the success or failure of certain strategies as the search progresses.
3. **Self-adaptive**: The algorithm adjusts its own parameters autonomously based on its performance.

- Effective parameter setting, whether off-line or on-line, can greatly enhance the performance of a metaheuristic algorithm, leading to faster convergence times and better solutions. It's often a balance between sufficient exploration of the search space and exploitation of good solutions found.

Parameter Tuning Methods

Traditional Approaches

- **Traditional Approaches**: These are the most straightforward methods for setting parameters. They may not be systematic and can rely heavily on the experience and expertise of the practitioner. This approach can be hit or miss and may not lead to the best performance.
- **Arbitrary Setting**: Choosing parameter values without systematic reasoning (Example: setting a parameter ϕ to 0 and another δ to 80).
- **Trial and Error**: Adjusting parameters based on intuition and observing the outcomes.
- **Theoretical Studies**: Utilising findings from theoretical research to inform parameter choices.
- **Mixed Approach**: Combining the above methods for parameter setting.

▼ **Advantages**:

- Quick to implement, especially if the user has prior experience or intuition about the parameters.
- May leverage existing theoretical work to guide choices.

▼ **Disadvantages**:

- Can be unsystematic and reliant on subjective judgement.
- May not yield the best possible outcomes due to lack of exhaustive exploration if the search space is massive.
- The success of the approach can be highly variable and not repeatable.

Sequential Tuning

- **Sequential Tuning**: This is a more systematic approach compared to traditional methods. By adjusting one parameter at a time, it is possible to see the individual effect of each parameter on the performance of the algorithm. This helps in understanding the parameter sensitivity and the interaction between different parameters. However, this method can be time-consuming, especially when the number of parameters is large.
- **Process**: Parameters are set and adjusted one after another in a sequence rather than all at once.

- **Example**: First, fix δ at 20 and try different values of ϕ such as {0, 0.3, 0.5, 0.8, 1.0}. After determining the best value for ϕ, proceed to tune δ with a range of values like {20, 40, 50, 60, 80}.

▼ **Advantages**:

- Allows for a clear understanding of how each parameter affects performance.
- Can identify the most sensitive parameters that have the greatest impact on outcomes.

▼ **Disadvantages**:

- Time-consuming as it requires multiple rounds of tuning.
- Does not account for interactions between parameters since they are tuned independently.

Design of Experiments

- **Design of Experiments**: This is a scientific approach that uses statistical techniques to design experiments that systematically investigate the parameter space. This method can help in understanding the interactions between different parameters and can also be used to find the combination of parameter settings that optimises the performance.
- This approach involves planning and conducting experiments to explore the parameters systematically, often guided by statistical principles.

▼ **Advantages**:

- Provides a structured and systematic approach to parameter tuning.
- Can reveal complex interactions between parameters.
- Utilizes statistical rigor to ensure the reliability of the results.

▼ **Disadvantages**:

- Requires statistical expertise to design and interpret experiments.
- Potentially resource-intensive in terms of the number of experiments needed.

Meta-optimisation

- **Meta-optimisation**: This is a sophisticated approach where a metaheuristic is used to tune the parameters of another metaheuristic. The metaheuristic that performs the tuning treats the parameter settings as solutions to be optimised and searches for the combination of settings that yields the best performance. This approach is powerful because it can automate the tuning process and potentially discover parameter settings that might not be found through manual tuning. However, it can also be computationally intensive.
- **Concept**: Utilising a metaheuristic algorithm to optimise the parameters of another metaheuristic.
- **Objective**: To find the "optimal" parameter settings that would result in the best performance of the algorithm.

▼ **Advantages**:

- Automates the parameter tuning process, reducing the need for human intervention.
- Can explore a large parameter space and potentially find better settings than manual tuning.

▼ **Disadvantages**:

- Computationally expensive as it involves running an optimization algorithm on top of the original problem.
- May be overkill for problems with a small number of parameters or where good parameter values are already known.

Automatic Parameter Tuning Methods

- Automated Parameter Tuning methods, are techniques used to automatically select the best parameters for a given algorithm or model.
- These methods can significantly improve the performance of algorithms without human intervention. The infographic categorises these methods into two main groups:

- Model-free
- Model-based.

Model-Free Methods:

- **Heuristic Search**: This involves search strategies that try to find a good enough solution when an exhaustive search is not feasible due to time constraints.
- **Meta EA (Evolutionary Algorithm)**: Uses evolutionary strategies to optimize the parameters of other evolutionary algorithms.
- **Meta GA (Genetic Algorithm)**: Uses genetic algorithms to tune the parameters of another genetic algorithm.
- **REVAC**: Stands for Relevance Estimation and Value Calibration, which estimates parameter importance and adjusts them accordingly.
- **GGA (Gender Genetic Algorithm)**: A variant of genetic algorithms that introduces the concept of "gender" to maintain diversity in the population.
- **ROAR**: Random Online Adaptive Racing, a simpler version of racing algorithms for parameter tuning.
- **FocussedILS and BasicILS (Iterated Local Search)**: These are iterative optimization algorithms that repeatedly apply local search to an evolving single candidate solution.
- **Adaptive Capping**: This is a technique that helps to speed up the tuning process by adaptively setting a cut-off time for poor parameter settings based on previous observations.
- **ParamILS**: An algorithm for optimizing parameters of iterative algorithms based on Iterated Local Search.

Model-Based Methods:

- **DoE (Design of Experiments)**: A systematic method to design experiments so that the data obtained can be analysed to yield valid and objective conclusions.

- **DACE (Design and Analysis of Computer Experiments)**: Uses statistical techniques to build a surrogate model of the objective function.
- **EGO (Efficient Global Optimisation)**: Combines global search with local search, based on the surrogate model usually created by DACE.
- **SKO (Sequential Kriging Optimisation)**: Utilises kriging models to sequentially find the optimal parameters.
- **SPO (Sequential Parameter Optimisation)**: Similar to SKO, but uses more general surrogate models.
- **SPO+ and TB-SPO**: Enhanced versions of SPO, where SPO+ includes mechanisms to deal with noise, and TB-SPO introduces trust bounds.
- **SMCA**: Sequential Model-based Algorithm Configuration, which combines ideas from SPO and competitive algorithm configuration.
- **Racing**: A family of methods that compare different parameter settings based on their performance on a subset of the problem instances and eliminate poor performers.
- **F-Race**: An extension of the racing method that focuses on discarding the inferior solutions.
- **Calibra**: A method that aims to improve the racing strategies by incorporating problem instance features.
- **Gaussian Process and Random Forests**: These are types of surrogate models used in model-based optimisation to predict the performance of parameter settings.

Design of Experiments

Concept:

What is design of experiments?

- Design of experiments (DOE) is a systematic, efficient method that enables scientists and engineers to study the relationship between multiple input

variables (aka factors) and key output variables (aka responses). It is a structured approach for collecting data and making discoveries.

- DoE is a series of tests or runs, called experiments, that allow you to systematically change one or more processes or system variables (factors) to observe the effect on the output (response).

When to use DOE?

- To determine whether a factor, or a collection of factors, has an effect on the response.
- To determine whether factors interact in their effect on the response.
- To model the behaviour of the response as a function of the factors.
- To optimise the response.
- Its a systematic method to determine the relationship between controllable and uncontrollable factors
- **Purpose**: The primary goal is to identify the relationship between factors and the response and to find the combination of factor levels that optimise the response.

Process:

- **Controlled Experiments**: Experiments where one or more factors are changed to determine their effect on the response. This change is conducted in a controlled manner to ensure that any variation in the response is due to the change in factors and not due to external variables.
- **Factors and Levels**: Factors are the inputs or conditions that are tested, and each factor has different settings or quantities, known as levels.
- **Response**: The output of the process, which can be the performance of an algorithm, quality of solutions, or any other measurable outcome.

Outcome Analysis:

- **Measurement**: Collecting data on the important outcomes or responses resulting from the experiments.

- **Analysis**: Using statistical methods to analyze the data to determine which factors and their levels lead to the best overall outcome.
- **Optimisation**: Identifying settings that provide the best performance or the most favorable output.

Example:

- Consider two factors, ϕ and δ, each with five levels. For ϕ, the levels are {0, 0.3, 0.5, 0.8, 1.0}, and for δ, the levels are {20, 40, 50, 60, 80}.
- To fully explore the interaction between these two factors, at least 5^2 (25) experimental runs are needed, assuming one run for each combination of factor levels.

Advantages of DoE:

- **Efficiency**: DoE can be more efficient than one-factor-at-a-time experiments because it can evaluate multiple factors and their interactions simultaneously.
- **Rigor**: Provides a rigorous framework to understand the effects of multiple factors on an outcome.
- **Predictive**: The statistical models developed from DoE can predict responses for a combination of factor levels, even those that have not been explicitly tested.

Considerations:

- **Fisher's Principle**: The methodology is rooted in principles established by Ronald Fisher in the early 20th century, emphasising randomisation, replication, and blocking to yield valid and objective conclusions.
- **Experiment Runs**: The number of runs required can grow quickly with the number of factors and levels, which might require significant resources and planning.

Fractional Factorial Designs

- A fractional factorial design is a type of experimental setup used in Design of Experiments (DOE) that allows researchers to study the effects of multiple

factors by conducting only a fraction of the total experiments that would be required for a full factorial design.

- **Full Factorial Design Recap:**
  - In a full factorial design, you would test all possible combinations of levels for all factors. For example, if you had 3 factors (A, B, C) each with 2 levels (low and high), a full factorial design would require 2^3 = 8 different experiments.
- **Fractional Factorial Design Explained:**
  - In many practical scenarios, especially when the number of factors increases, the number of required experiments for a full factorial design grows exponentially and becomes unfeasible.
  - A fractional factorial design uses only a subset of the full factorial experiment combinations to estimate the main effects and possibly some of the interaction effects.
  - This is done by carefully selecting a fraction of the full factorial design such that the most information can be gained with the least amount of experimental effort.
  - For example, instead of running all 8 experiments, you might run only 4, carefully chosen to capture the most important information about the main effects and interactions.
- **Relation to Design of Experiments/Parameter Tuning:**
  - In parameter tuning, especially when dealing with complex models (like in machine learning), a fractional factorial design can help you determine which parameters (ingredients) are most important to tune and which combinations lead to the best performance (tastiest cake) without having to exhaustively try every possible parameter combination. It's a strategy to get the most learning with the least amount of effort.

Concept:

- **Full Factorial Design Issue**: In a full factorial design, the number of experimental runs needed can become very large as the number of factors increases, especially if each factor has multiple levels.

- **Mathematical Notation**: The total number of runs required for a full factorial design with k factors at n levels each is n^k.

Example:

- **Full Experiment**: For an experiment with 8 factors each at 2 levels (a 2-level factorial design), you would need $2^8 = 256$ runs.
- **Time Consideration**: If each run takes 5 minutes, a single replicate of this full factorial design would take 256×5 minutes, or approximately 28 days for a single instance.

Analogy:

- Imagine you're trying to find the perfect recipe for a cake, and you have several ingredients to experiment with, such as flour type, sugar level, egg size, and oven temperature. A full factorial design would mean baking a cake for every possible combination of these ingredients—this would be quite a lot of baking!
- Instead, you opt for a fractional factorial design. Think of it as reading online reviews for cakes. You don't try every possible recipe; you read a selection of reviews that cover a range of combinations. From these reviews, you can infer which ingredients and temperatures tend to result in a good cake and which don't. You don't have the full picture, but you have enough to make an educated guess about a good recipe.
- In this analogy, each review represents an experiment in the fractional factorial design. By selecting a strategic subset of "reviews" (experiments), you can learn about the main factors that contribute to a delicious cake without having to bake every possible variation.

Key Observations:

- **Main Effects vs. Interactions**: Responses in an experiment are often more influenced by the main effects of factors and their low-order interactions. High-order interactions (interactions involving many factors) are typically less significant.

- **Efficiency**: By focusing on the main effects and lower-order interactions, fractional factorial designs provide a more efficient and practical approach to experimentation.

▼ **Advantages:**

- Requires fewer experiments, saving time and resources.
- Can still provide insight into the most significant factors and interactions.

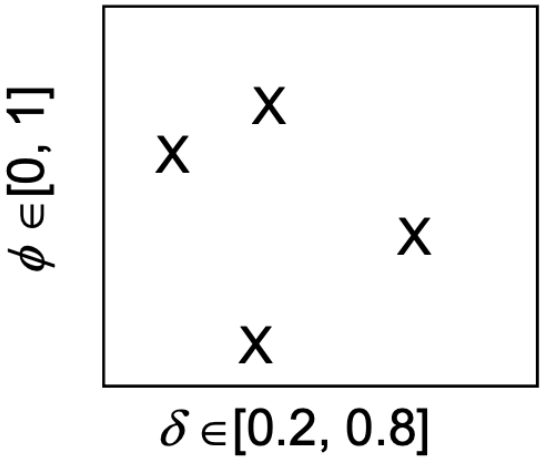
▼ **Disadvantages:**

- You may miss some higher-order interactions because not all combinations are tested.
- The resolution of the design decreases, which means that some effects may be confounded with others (i.e., it may be difficult to distinguish between the effects of different factors).

Design of Experiments - Sampling

- Design of Experiments (DOE) is a systematic method to determine the relationship between factors affecting a process and the output of that process. In cases where a full factorial design is not feasible due to constraints like time, cost, or the sheer number of experiments that would be required, sampling methods are employed to efficiently explore the space of parameters.

1. **Random Sampling:**
    - **Explanation**: This is the simplest form of sampling where each sample point is generated independently and randomly within the specified range of parameters.



▼ **Advantages**:

- Easy to implement.
- Requires no prior knowledge of the system.

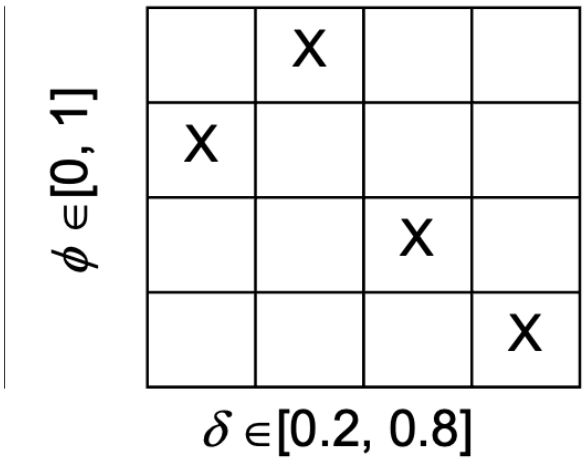
▼ **Disadvantages**:

- Inefficient for high-dimensional problems as it may require a large number of samples to explore the space adequately.
- There is a possibility of clustering of points which can leave large regions of the space unsampled.

- **Relation to DOE/Parameter Tuning**: Random sampling can be a starting point for parameter tuning but might not be efficient for complex models or when the parameter space is large.

2. **Latin Hypercube Sampling (LHS):**

- **Explanation**: This method involves dividing the range of each parameter into N intervals of equal probability, and then sample points are selected in a way that each interval is sampled only once. This ensures that the entire range of each parameter is explored.



▼ **Advantages**:

- More efficient than random sampling in covering the range of parameters.
- Requires fewer samples to explore the space adequately compared to random sampling.

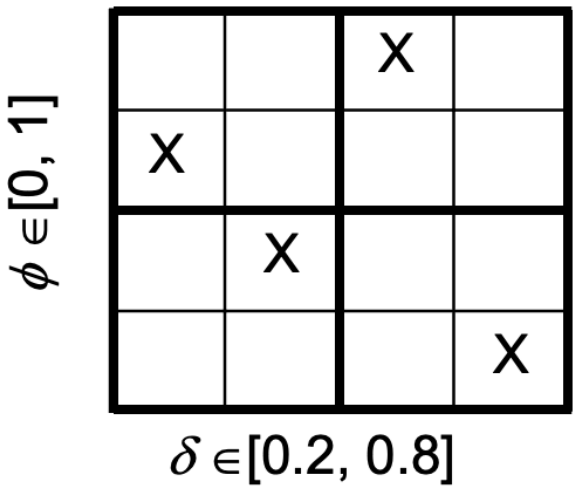
▼ **Disadvantages**:

- Can be more complex to implement than random sampling.
- Does not guarantee that the sample points will be maximally spread out.

- **Relation to DOE/Parameter Tuning**: LHS is used when it is important to ensure that the entire parameter space is explored. It is particularly useful when each parameter is equally important, and we want to avoid the risk of missing out on any particular region of the parameter space.

3. **Orthogonal Sampling:**

- **Explanation**: This technique extends Latin Hypercube Sampling by not only ensuring that each interval is sampled once, but also that the sample points are as spread out as possible, following the principles of orthogonal arrays based on regions.



▼ **Advantages**:

- Ensures a more uniform sampling of the space than LHS.
- Helps in reducing the variance of the estimates of the model's output.

▼ **Disadvantages**:

- More complex to set up than LHS and random sampling.
- The construction of orthogonal arrays can be challenging, especially for a large number of parameters or levels.

- **Relation to DOE/Parameter Tuning**: Orthogonal sampling is highly effective in parameter tuning for complex simulations and models, where interactions between parameters are significant. It helps in understanding the effects of multiple parameters on the response variable without the need for a full factorial design.

- In summary, the choice of sampling method in DOE depends on the specific needs of the experiment, such as the number of parameters, the interactions between them, and the resources available.
- Random sampling is quick and simple, but not as efficient for complex or high-dimensional problems.
- Latin Hypercube Sampling offers a good balance between simplicity and efficiency.

- While Orthogonal Sampling provides the most thorough exploration of the parameter space but requires more effort to set up.

The Taguchi Method

- The **Taguchi Method**, developed by Genichi Taguchi, is a statistical method sometimes referred to as robust design methods.
- It was initially created to improve the quality of manufactured goods but has since been applied to various fields such as engineering, biotechnology, marketing, and advertising.
- The method aims to reduce variation in a process through robust design of experiments. The overall objective is to produce high-quality products at a low cost to the manufacturer.
- Basically make a “product” or “process” less variable (more *robust*) in the face of variation over which we have little or no control.
- lTaguchi method is a structured statistical (experimental design) method for determining the ***best*** combination of parameter settings to achieve certain objective(s)

Taguchi Method Explained

1. **Objective**:
    - Improve the quality of products/processes.
    - Reduce variations in processes.
    - Minimise cost while maximising quality.
2. **Orthogonal Arrays**:
    - A key tool in the Taguchi Method.
    - Used for designing experiments to test multiple variables efficiently.
    - Enables testing fewer combinations than a full factorial design (which tests all possible combinations).
3. **Efficiency**:

- Instead of testing every single possible combination (which can be extensive), Taguchi uses orthogonal arrays to test pairs of combinations.
- This method saves time and resources by minimising the number of experiments.

4. **Ideal Conditions**:
   - Best for 3 to 50 variables.
   - Works well when few interactions exist between variables.
   - Most beneficial when only a few variables significantly affect the outcome.
5. **Process**:
   - Variables (factors) are assigned levels, typically "Low" (L) and "High" (H).
   - An orthogonal array is selected to set up experiments where these factors are varied systematically.
   - Each combination is tested to see the effect on the outcome.
6. **Analysis**:
   - After conducting experiments, an Analysis of Variance (ANOVA) can determine which factors and levels have the most significant impact.
   - Use this analysis to optimise parameters for the best performance.

Example Breakdown

- **Variables**: A, B, C
- **Levels**: Low (L), High (H)
- **Traditional Approach**:
  - Would require \(2^3 = 8\) experiments for all combinations.
- **Taguchi's Approach**:
  - Only requires 4 experiments using an orthogonal array.

Score Calculation Example

- You assign scores to each level of the variables based on the results of the experiments.
- For instance:
  - A = L; Score 99
  - A = H; Score 98
  - B = L; Score 89
  - B = H; Score 79
  - C = L; Score 88
  - C = H; Score 85
- **Interpretation**:
  - The scores represent the impact of each level of the variable on the outcome.
  - Since each level is tested an equal number of times, the effect of other variables is neutralised.
  - You can determine which variable level combination gives you the best score, indicating the least variability and optimal performance.

Main Steps in Taguchi Method

1. **Selection of control parameters**: Identify the independent variables or factors that are to be tested.
2. **Selection of the number of level settings for each parameter**: Decide how many variations of each factor will be tested, such as high or low temperature.
3. **Select a suitable orthogonal array**: Based on the number of parameters and the levels, choose an array that minimises the number of experimental runs but still captures the essential information about the parameter effects.
4. **Conduct the experiments**: Perform the experiments using the chosen orthogonal array. This focuses on a subset of all possible test instances, but it is designed to extract maximum information.

5. **Analyse the results**: After conducting the experiments, analyse the data to determine the effects of the different factors and their levels.
6. **Determine the optimum levels for the individual parameters**: Based on the analysis, identify which levels of each factor lead to the best performance.
7. **Confirmation experiment**: Perform an experiment using the identified optimal levels to verify the results obtained are consistent and repeatable.

Planning Phase

- Lets assume we have a problem and we are designing a metaheuristic to solve this problem.
- We want to first follow the 3 general steps (from above):

1. Selection of control parameters (independent variables).
2. Selection of number of level settings for each parameter.
3. Select a suitable ornithological array based on the number of parameters and levels.

- We have 4 parameters of the algorithm:
  - $param1 \in [0, 1.0](\mathbb{R})$ - parameter 1 real values that change between 0 and 1
  - $param2 \in [0, 1.0](\mathbb{R})$ - parameter 2 real values that change between 0 and 1
  - $param3 \in [1..80](\mathbb{Z}^+)$ - parameter 3 integer values that change between 1 and 80
  - $param4 \in [1..5](\mathbb{Z}^+)$ - parameter 4 integer values that change between 1 and 5
- We then continue following the basic outline of the steps and select levels for each parameter.

Levels	Param	Value Options
5	param1	{0.2, 0.4, 0.6, 0.8, 1.0}

Levels	Param	Value Options
5	param2	{0.2, 0.4, 0.6, 0.8, 1.0}
5	param3	{5, 10, 20, 40, 80}
4	param4	{2, 3, 4, 5}

- Here we have 4 factors with a maximum of 5 levels and we will enter this into the system.
- The Taguchi ornithological array may suggest for example 25 combinations of parameter settings:

Taguchi orthogonal array

Configuration ID		param1	param2	param3	param4
1	e.g.;	1 (0.2)	1 (0.2)	1 (5)	1 (2)
2		1	2	2	2
3		1	3	3	3
4		1	4	4	4
5		1	5	5	2
6		2	1	2	3
7		2	2	3	4
8		2	3	4	1
9		2	4	5	1
10		2	5	1	2
11		3	1	3	4
12		3	2	4	1
13		3	3	5	2
14		3	4	1	3
15		3	5	2	4
16		4	1	4	2
17		4	2	5	3
18		4	3	1	4
19		4	4	2	2
20		4	5	3	1
21		5	1	5	4
22		5	2	1	3
23		5	3	2	1
24		5	4	3	2
25	e.g.;	5 (1.0)	5 (1.0)	4 (40)	3 (4)



- The array above ID 1 for example suggests to use the 1st of each parameter setting to conduct the experiment with.

Conducting Experiments Phase

Configuration ID	param1	param2	param3	param4
1	0.2	0.2	5	2
2	0.2	0.4	10	3
3	0.2	0.6	20	4
4	0.2	0.8	40	5
5	0.2	1.0	80	3
6	0.4	0.2	10	4
7	0.4	0.4	20	5
8	0.4	0.6	40	2
9	0.4	0.8	80	2
10	0.4	1.0	5	3
11	0.6	0.2	20	5
12	0.6	0.4	40	2
13	0.6	0.6	80	3
14	0.6	0.8	5	4
15	0.6	1.0	10	5
16	0.8	0.2	40	3
17	0.8	0.4	80	4
18	0.8	0.6	5	5
19	0.8	0.8	10	3
20	0.8	1.0	20	2
21	1.0	0.2	80	5
22	1.0	0.4	5	4
23	1.0	0.6	10	2
24	1.0	0.8	20	3
25	1.0	1.0	40	4

55

- We then take the table and convert it into our actual parameter settings and run experiments on them.

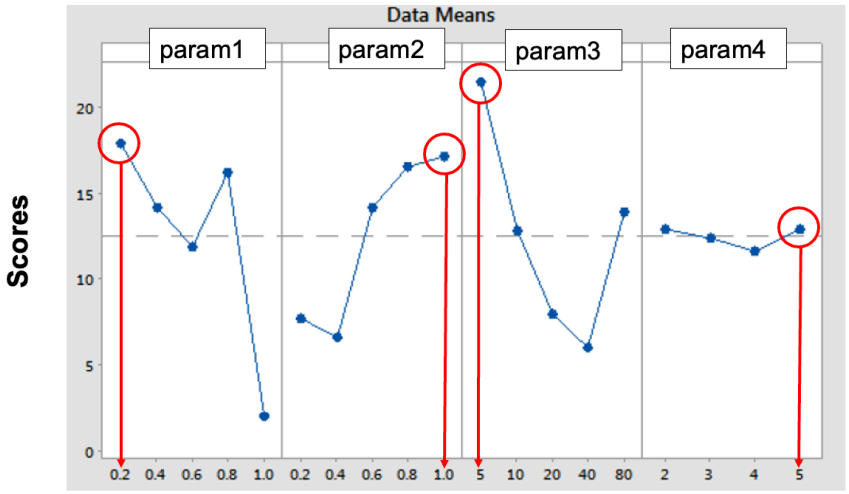
- Then run experiments, say for 30 times using the algorithm with each setting (potentially on multiple 'training' instances)
- Use a performance metric, e.g., record Formula 1 score for each run/trial (higher the better), where the top 8 algorithms score **10, 8, 6, 5, 4, 3, 2** and **1** point(s).
- We then have to collect/measure the results

Configuration ID	param1	param2	param3	param4	Performance Indicator/Scores
1	0.2	0.2	5	2	24.88
2	0.2	0.4	10	3	14
3	0.2	0.6	**20**	4	**17.50**
4	0.2	0.8	40	5	17.88
5	0.2	1.0	80	3	15.50
6	0.4	0.2	10	4	9.88
7	0.4	0.4	**20**	5	**2.50**
8	0.4	0.6	40	2	8.88
9	0.4	0.8	80	2	22.50
10	0.4	1.0	5	3	27.25
11	0.6	0.2	**20**	5	**0**
12	0.6	0.4	40	2	1
13	0.6	0.6	80	3	12
14	0.6	0.8	5	4	24.38
15	0.6	1.0	10	5	22.25
16	0.8	0.2	40	3	0
17	0.8	0.4	80	4	25.88
18	0.8	0.6	5	5	30.88
19	0.8	0.8	10	3	16.25
20	0.8	1.0	**20**	2	**18.38**
21	1.0	0.2	80	5	4
22	1.0	0.4	5	4	0
23	1.0	0.6	10	2	1.88
24	1.0	0.8	**20**	3	**1.88**
25	1.0	1.0	40	4	2.50

- Collect results using an appropriate performance indicator: e.g., obtain mean F1 *score* per run for each algorithm with a specific parameter combination setting per instance, and sum those scores up from all instances

- For analysis: Main effect of param3 for the setting of 20 is the *mean score* for all experiments with that setting: (17.5+2.5+0+18.38+1.88)/5 =8.052

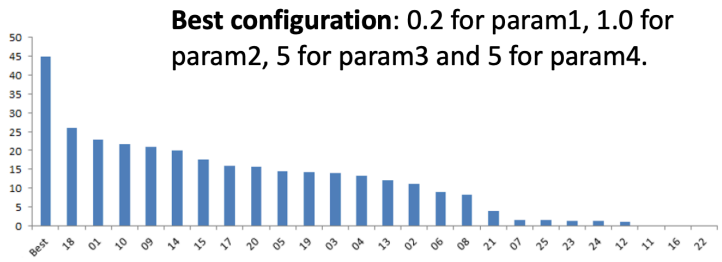
Analysis and Validation Phase



- We pick out the max for each (maximisation problem instance) and these are the parameters we should be using
  - Param 1 - 0.2
  - Param 2 - 1.0
  - Param 3 - 5
  - Param 4 - 5

Validation: Confirmation Run





If failure, then chosen levels could be problematic needs to be reviewed.

Soft Drink Bottling Problem

(need to write)

https://prod-files-secure.s3.us-west-2.amazonaws.com/259ac55a-7b50-42c6-b172-d031975b9e58/8bf28738-8828-464d-bd1a-4302db307679/4.3._QUIZ_-_Metaheuristics_and_Move_Acceptance_(Lecture_4)_Self-assessment_Activity__Attempt_review___moodle.Nottingham.pdf